DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

AD3405 – SOFTWARE ENGINEERING

UNIT I - SOFTWARE PROCESS

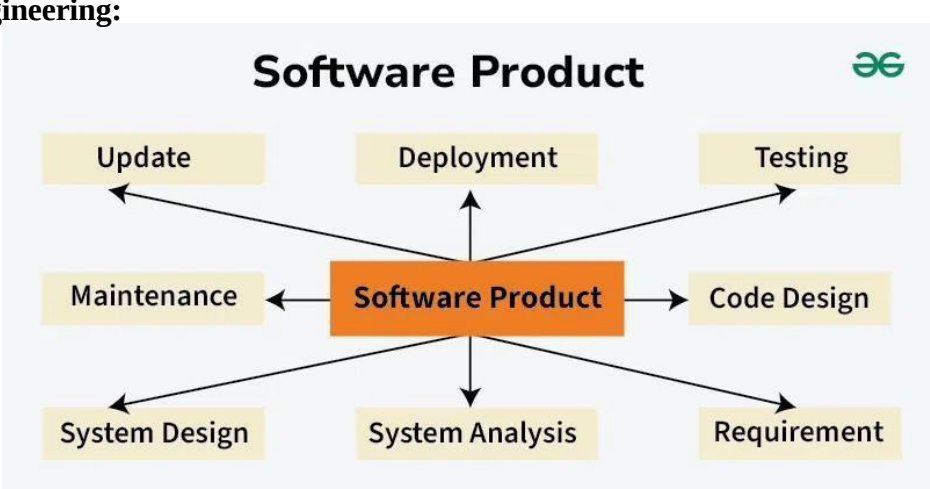
Introduction to software Engineering - Software Process- Perspective Models: Waterfall, Iterative & Incremental, V – and Spiral Models – Specialized Process Models : Agile Models, Component Based Development (CBD), Rapid Application Development (RAD), and Formal Methods

---

What is Software?

Software is a program or set of programs containing instructions that provide the desired functionality. Engineering is the process of designing and building something that serves a particular purpose and finds a cost-effective solution to problems.

Software Engineering:



Software Engineering is the process of designing, developing, testing, and maintaining software. It is a systematic and disciplined approach to software development that aims to create high-quality, reliable, and maintainable software.

1. Software engineering includes a variety of techniques, tools, and methodologies, including requirements analysis, design, testing, and maintenance.
2. It is a rapidly evolving field, and new tools and technologies are constantly being developed to improve the software development process.
3. By following the principles of software engineering and using the appropriate tools and methodologies, software developers can create high-quality, reliable, and maintainable software that meets the needs of its users.
4. Software Engineering is mainly used for large projects based on software systems rather than single programs or applications.
5. The main goal of Software Engineering is to develop software applications for improving quality, budget, and time efficiency.
6. Software Engineering ensures that the software that has to be built should be consistent, correct, also on budget, on time, and within the required requirements.

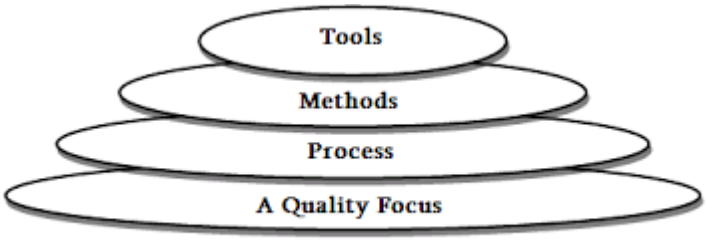


Fig: Software Engineering Layers

Key Principles of Software Engineering

1. **Modularity**: Breaking the software into smaller, reusable components that can be developed and tested independently.
2. **Abstraction**: Hiding the implementation details of a component and exposing only the necessary functionality to other parts of the software.
3. **Encapsulation**: Wrapping up the data and functions of an object into a single unit, and protecting the internal state of an object from external modifications.
4. **Reusability**: Creating components that can be used in multiple projects, which can save time and resources.
5. **Maintenance**: Regularly updating and improving the software to fix bugs, add new features, and address security vulnerabilities.
6. **Testing**: Verifying that the software meets its requirements and is free of bugs.
7. **Design Patterns**: Solving recurring problems in software design by providing templates for solving them.
8. **Agile methodologies:** Using iterative and incremental development processes that focus on customer satisfaction, rapid delivery, and flexibility.
9. **Continuous Integration & Deployment:** Continuously integrating the code changes and deploying them into the production environment.

Main Attributes of Software Engineering

Software Engineering is a systematic, disciplined, quantifiable study and approach to the design, development, operation, and maintenance of a software system. There are four main Attributes of Software Engineering.

1. **Efficiency:** It provides a measure of the resource requirement of a software product efficiently.
2. **Reliability:** It assures that the product will deliver the same results when used in similar working environment.
3. **Reusability:** This attribute makes sure that the module can be used in multiple applications.
4. **Maintainability:** It is the ability of the software to be modified, repaired, or enhanced easily with changing requirements.

Need of Software Engineering:

- **Handling Big Projects:** A corporation must use a software engineering methodology in order to handle large projects without any issues.

- **To manage the cost:** Software engineering programmers plan everything and reduce all those things that are not required.
- **To decrease time:** It will save a lot of time if you are developing software using a software engineering technique.
- **Reliable software:** It is the company's responsibility to deliver software products on schedule and to address any defects that may exist.
- **Effectiveness:** Effectiveness results from things being created in accordance with the standards.
- **Reduces complexity:** Large challenges are broken down into smaller ones and solved one at a time in software engineering. Individual solutions are found for each of these issues.
- **Productivity:** Because it contains testing systems at every level, proper care is done to maintain software productivity.

Tasks :

The main responsibility of a software engineer is to develop useful computer programs and applications. Working in teams, you would complete various projects and develop solutions to satisfy certain customer or corporate demands.

Some of the key responsibilities of software engineer are:

- **Requirement Analysis**: Collaborating with stakeholders to understand and gather the requirements to design and develop software solutions.
- **Design and Development**: Creating well-structured, maintainable code that meets the functional requirements and adheres to software design principles.
- **Testing and Debugging**: Writing and conducting unit tests, integration tests, and debugging code to ensure software is reliable and bug-free.
- **Code Review**: Participating in code reviews to improve code quality, ensure adherence to standards, and facilitate knowledge sharing among team members.
- **Maintenance**: Updating and maintaining existing software systems, fixing bugs, and improving performance or adding new features.
- **Documentation**: Writing clear documentation, including code comments, API documentation, and design documents to help other engineers and future developers understand the system.

Advantages of Software Engineering

There are several advantages to using a systematic and disciplined approach to software development, such as:

1. **Improved Quality:** By following established software engineering principles and techniques, the software can be developed with fewer bugs and higher reliability.
2. **Increased Productivity:** Using modern tools and methodologies can streamline the development process, allowing developers to be more productive and complete projects faster.
3. **Better Maintainability:** Software that is designed and developed using sound software engineering practices is easier to maintain and update over time.

4. **Reduced Costs:** By identifying and addressing potential problems early in the development process, software engineering can help to reduce the cost of fixing bugs and adding new features later on.
5. I**ncreased Customer Satisfaction:** By involving customers in the development process and developing software that meets their needs, software engineering can help to increase customer satisfaction.
6. **Better Team Collaboration:** By using Agile methodologies and continuous integration, software engineering allows for better collaboration among development teams.
7. **Better Scalability**: By designing software with scalability in mind, software engineering can help to ensure that software can handle an increasing number of users and transactions.
8. **Better Security:** By following the Software Development Life Cycle (SDLC) and performing security testing, software engineering can help to prevent security breaches and protect sensitive data.

Software Processes:

Software processes in software engineering refer to the methods and techniques used to develop and maintain software. Some examples of software processes include:

- Waterfall: a linear, sequential approach to software development, with distinct phases such as requirements gathering, design, implementation, testing, and maintenance.
- Agile: a flexible, iterative approach to software development, with an emphasis on rapid prototyping and continuous delivery.
- Scrum: a popular Agile methodology that emphasizes teamwork, iterative development, and a flexible, adaptive approach to planning and management.
- DevOps: a set of practices that aims to improve collaboration and communication between development and operations teams, with an emphasis on automating the software delivery process.

Software Process Model

A software process model is an abstraction of the actual process, which is being described. It can also be defined as a simplified representation of a software process. Each model represents a process from a specific perspective.

Following are some basic software process models on which different type of software process models can be implemented:

1. **A workflow Model :** It is the sequential series of tasks and decisions that make up a business process.
2. **The Waterfall Model:** It is a sequential design process in which progress is seen as flowing steadily downwards.
    - Phases in waterfall model:
        - Requirements Specification
        - Software Design
        - Implementation
        - Testing

3. **Dataflow Model:** It is diagrammatic representation of the flow and exchange of information within a system.
4. **Evolutionary Development Model:** Following activities are considered in this method:
    - Specification
    - Development
    - Validation
5. **Role / Action Model:** Roles of the people involved in the software process and the activities.

Need for Process Model

The software development team must decide the process model that is to be used for software product development and then the entire team must adhere to it. This is necessary because the software product development can then be done systematically. Each team member will understand what is the next activity and how to do it. Thus process model will bring the definiteness and discipline in overall development process. Every process model consists of definite entry and exit criteria for each phase. Hence the transition of the product through various phases is definite.

If the process model is not followed for software development then any team member can perform any software development activity, this will ultimately cause a chaos and software project will definitely fail without using process model, it is difficult to monitor the progress of software product. Thus process model plays an important rule in software engineering.

Advantages or Disadvantages of Process Model

There are several advantages and disadvantages to different software development methodologies, such as:

Waterfall

Advantages of waterfall model are:

1. Clear and defined phases of development make it easy to plan and manage the project.
2. It is well-suited for projects with well-defined and unchanging requirements.

Disadvantages of waterfall model are:

1. Changes made to the requirements during the development phase can be costly and time-consuming.
2. It can be difficult to know how long each phase will take, making it difficult to estimate the overall time and cost of the project.
3. It does not have much room for iteration and feedback throughout the development process.

Agile

Advantages of Agile Model are:

1. Flexible and adaptable to changing requirements.
2. Emphasizes rapid prototyping and continuous delivery, which can help to identify and fix problems early on.
3. Encourages collaboration and communication between development teams and stakeholders.

Disadvantages of Agile Model are:

1. It may be difficult to plan and manage a project using Agile methodologies, as requirements and deliverables are not always well-defined in advance.
2. It can be difficult to estimate the overall time and cost of a project, as the process is iterative and changes are made throughout the development.

Scrum

Advantages of Scrum are:

1. Encourages teamwork and collaboration.
2. Provides a flexible and adaptive framework for planning and managing software development projects.
3. Helps to identify and fix problems early on by using frequent testing and inspection.

Disadvantages of Scrum are:

1. A lack of understanding of Scrum methodologies can lead to confusion and inefficiency.
2. It can be difficult to estimate the overall time and cost of a project, as the process is iterative and changes are made throughout the development.

DevOps

Advantages of DevOps are:

1. Improves collaboration and communication between development and operations teams.
2. Automates software delivery process, making it faster and more efficient.
3. Enables faster recovery and response time in case of issues.

Disadvantages of DevOps are:

1. Requires a significant investment in tools and technologies.
2. Can be difficult to implement in organizations with existing silos and lack of culture of collaboration.
3. Need to have a skilled workforce to effectively implement the devops practices.
4. Ultimately, the choice of which methodology to use depends on the specific project and organization, as well as the goals and requirements of the project.

What are the four 4 main activities of the software process?

The four main activities of Software Process are:

1. Specification
2. Design and Development
3. Verification and Validation
4. Evolution

What are the three types of software process models?

The three main software processes models are:

1. Waterfall Model
2. Agile Model
3. Spiral Model

Explain **the Software Process Framework** in detail.

Framework is a Standard way to build and deploy applications. Software Process Framework is a foundation of complete software engineering process. Software process framework includes all set

of umbrella activities. It also includes number of framework activities that are applicable to all software projects.

A generic process framework encompasses five activities which are given below one by one:

Communication:

In this activity, heavy communication with customers and other stakeholders, requirement gathering is done.

Planning:

In this activity, we discuss the technical related tasks, work schedule, risks, required resources etc.

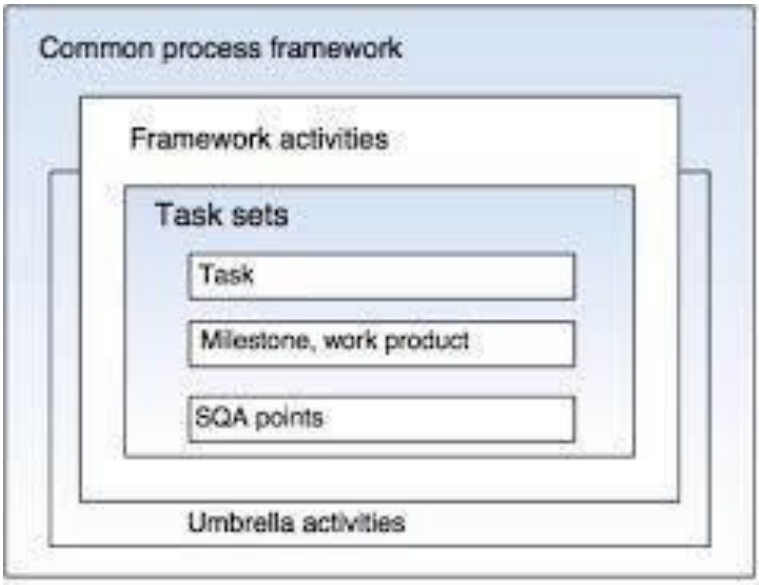


Fig.- A software process framework

Modeling:

Modelling is about building representations of things in the 'real world'.In modelling activity, a product's model is created in order to better understanding and requirements.

Construction:

In software engineering, construction is the application of set of procedures that are needed to assemble the product. In this activity, we generate the code and test the product in order to make better product.

Deployment: In this activity, a complete or non-complete products or software are represented to the customers to evaluate and give feedback. on the basis of their feedback we modify the products for supply better product.

Typical umbrella activities are:

1. Software project tracking and control

- In this activity, the developing team accesses project plan and compares it with the predefined schedule.
- If these project plans do not match with the predefined schedule, then the required actions are taken to maintain the schedule.

2. Risk management

- Risk is an event that may or may not occur.
- If the event occurs, then it causes some unwanted outcome. Hence, proper risk management is required.

3. Software Quality Assurance (SQA)

- SQA is the planned and systematic pattern of activities which are required to give a guarantee of software quality. For example, during the software development meetings are conducted at every stage of development to find out the defects and suggest improvements to produce good quality software.

4. Formal Technical Reviews (FTR)

- FTR is a meeting conducted by the technical staff.
- The motive of the meeting is to detect quality problems and suggest improvements.
- The technical person focuses on the quality of the software from the customer point of view. 5. Measurement
- Measurement consists of the effort required to measure the software.
- The software cannot be measured directly. It is measured by direct and indirect measures.
- Direct measures like cost, lines of code, size of software etc.
- Indirect measures such as quality of software which is measured by some other factor. Hence, it is an indirect measure of software.

6. Software Configuration Management (SCM)

It manages the effect of change throughout the software process.

7.Reusability management

- It defines the criteria for reuse the product.
- The quality of software is good when the components of the software are developed for certain application and are useful for developing other applications.

8. Work product preparation and production

- It consists of the activities that are needed to create the documents, forms, lists, logs and user manuals for developing a software.

Software Process Models / Prescriptive Models

Prescriptive process models advocate an orderly approach to software engineering

A software process model is a simplified representation of a software process. Each model represents a process from a specific perspective.

Some methodologies are sometimes known as **software development life cycle** (SDLC) methodologies, though this term could also be used more generally to refer to any methodology.

What is the SDLC Waterfall Model?

The waterfall model is a software development model used in the context of large, complex projects, typically in the field of information technology. It is characterized by a structured, sequential approach to project management and software development.

The waterfall model is useful in situations where the project requirements are well-defined and the project goals are clear. It is often used for large-scale projects with long timelines, where there

is little room for error and the project stakeholders need to have a high level of confidence in the outcome.

The name 'prescriptive' is given because the model prescribes a set of activities, actions, tasks, quality assurance and change the mechanism for every project.

3 types of Perspective models are:

1. TheWaterfallModel
2. TheIncrementalProcessmodel
3. RAD model

Features :

Following are the features of the waterfall model:

1. **Sequential Approach**: The waterfall model involves a sequential approach to software development, where each phase of the project is completed before moving on to the next one.
2. **Document-Driven:** The waterfall model depended on documentation to ensure that the project is well-defined and the project team is working towards a clear set of goals.
3. **Quality Control:** The waterfall model places a high emphasis on quality control and testing at each phase of the project, to ensure that the final product meets the requirements and expectations of the stakeholders.
4. **Rigorous Planning**: The waterfall model involves a careful planning process, where the project scope, timelines, and deliverables are carefully defined and monitored throughout the project lifecycle.

The waterfall model is used in situations where there is a need for a highly structured and systematic approach to software development. It can be effective in ensuring that large, complex projects are completed on time and within budget, with a high level of quality and customer satisfaction.

Importance of Waterfall Model :

Following are the importance of waterfall model:

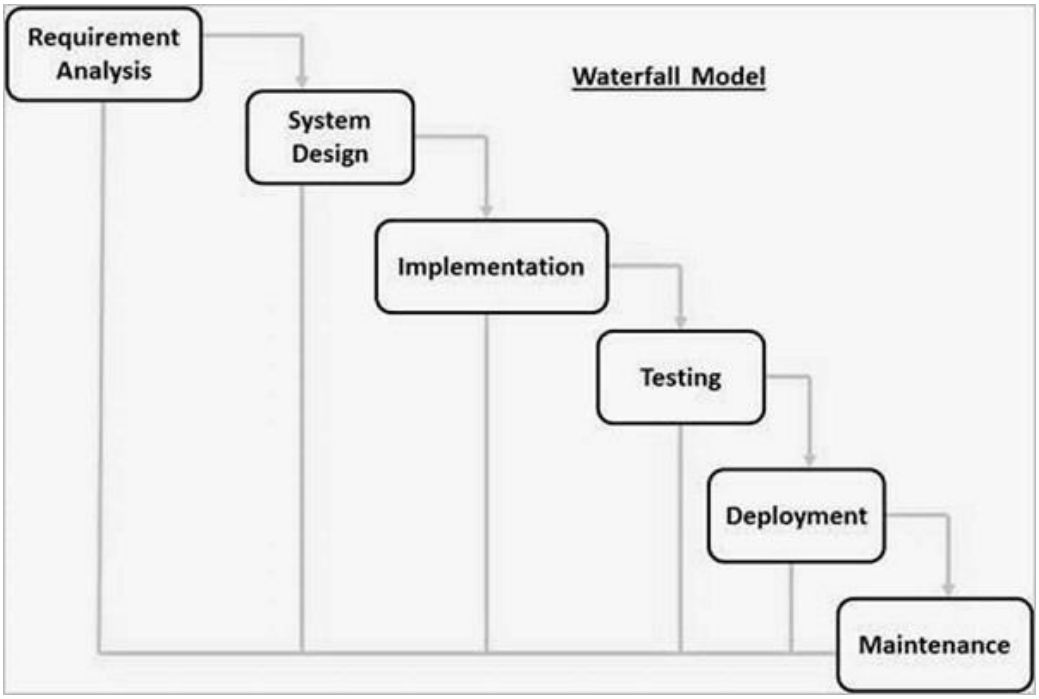
1. **Clarity and Simplicity:** The linear form of the Waterfall Model offers a simple and unambiguous foundation for project development.
2. **Clearly Defined Phases:** The Waterfall Model phases each have unique inputs and outputs, guaranteeing a planned development with obvious checkpoints.
3. **Documentation:** A focus on thorough documentation helps with software comprehension, maintenance, and future growth.
4. **Stability in Requirements:** Suitable for projects when the requirements are clear and stable, reducing modifications as the project progresses.
5. **Resource Optimization:** It encourages effective task-focused work without continuously changing contexts by allocating resources according to project phases.
6. **Relevance for Small Projects:** Economical for modest projects with simple specifications and minimal complexity.

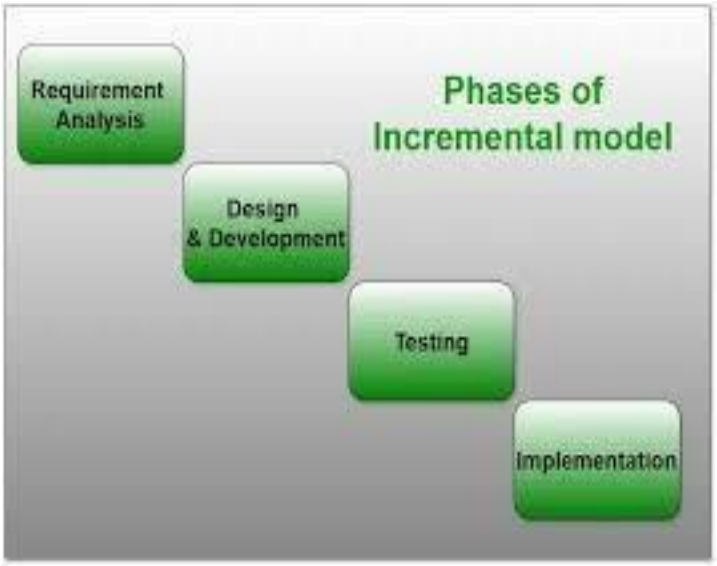
Phases of Waterfall Model

The Waterfall Model has six phases which are:

1. **Requirements:** The first phase involves gathering requirements from stakeholders and analyzing them to understand the scope and objectives of the project.
2. **Design:** Once the requirements are understood, the design phase begins. This involves creating a detailed design document that outlines the software architecture, user interface, and system components.
3. **Development:** The Development phase include implementation involves coding the software based on the design specifications. This phase also includes unit testing to ensure that each component of the software is working as expected.
4. **Testing:** In the testing phase, the software is tested as a whole to ensure that it meets the requirements and is free from defects.
5. **Deployment:** Once the software has been tested and approved, it is deployed to the production environment.
6. **Maintenance:** The final phase of the Waterfall Model is maintenance, which involves fixing any issues that arise after the software has been deployed and ensuring that it continues to meet the requirements over time.

The classical waterfall model divides the life cycle into a set of phases. This model considers that one phase can be started after the completion of the previous phase. That is the output of one phase will be the input to the next phase. Thus the development process can be considered as a sequential flow in the waterfall. Here the phases do not overlap with each other. The different sequential phases of the classical waterfall model are shown in the below figure.





Incremental Development Model

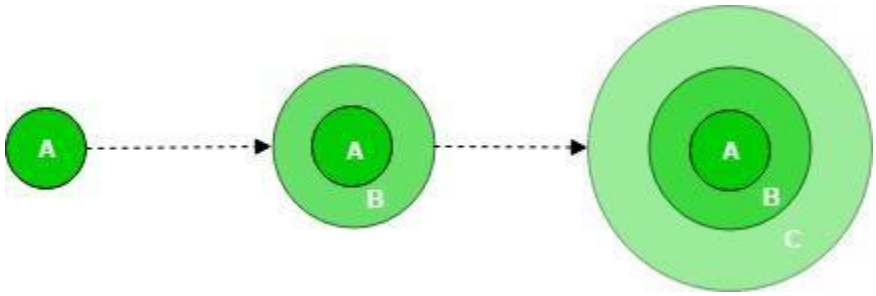
What is the Incremental Process Model?

First, a simple working system implementing only a few basic features is built and then that is delivered to the customer. Then thereafter many successive iterations/ versions are implemented and delivered to the customer until the desired system is released.

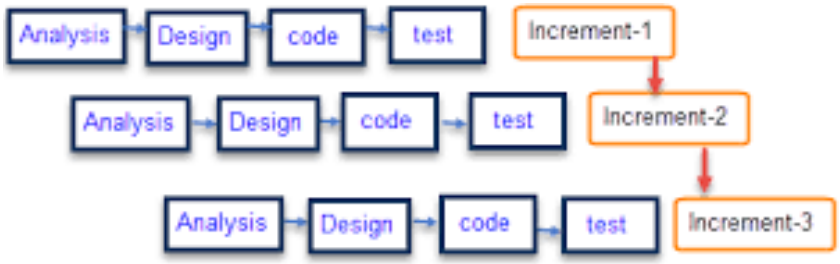
1. **Requirement analysis:** In Requirement Analysis At any time, the plan is made just for the next increment and not for any kind of long-term plan. Therefore, it is easier to modify the version as per the needs of the customer.
2. **Design & Development:** At any time, the plan is made just for the next increment and not for any kind of long-term plan. Therefore, it is easier to modify the version as per the needs of the customer. The Development Team first undertakes to develop core features (these do not need services from other features) of the system. Once the core features are fully developed, then these are refined to increase levels of capabilities by adding new functions in Successive versions. Each incremental version is usually developed using an iterative waterfall model of development.
3. **Deployment and Testing:** After Requirements gathering and specification, requirements are then split into several different versions starting with version 1, in each successive increment, the next version is constructed and then deployed at the customer site. in development and Testing the product is checked and tested for the actual process of the model.
4. **Implementation:** In implementation After the last version (version n), it is now deployed at the client site.

A, B, and C are modules of Software Products that are incrementally developed and delivered.

Phases of incremental model



Requirements of Software are first broken down into several modules that can be incrementally constructed and delivered.



Incremental Model

Characteristics of Incremental Process Model

1. System development is divided into several smaller projects.
2. To create a final complete system, partial systems are constructed one after the other.
3. Priority requirements are addressed first.
4. The requirements for that increment are frozen once they are created.

Advantages of the Incremental Process Model

1. Prepares the software fast.
2. Clients have a clear idea of the project.
3. Changes are easy to implement.
4. Provides risk handling support, because of its iterations.
5. Adjusting the criteria and scope is flexible and less costly.
6. Comparing this model to others, it is less expensive.
7. The identification of errors is simple.

Disadvantages of the Incremental Process Model

1. A good team and proper planned execution are required.
2. Because of its continuous iterations the cost increases.
3. Issues may arise from the system design if all needs are not gathered upfront throughout the program lifecycle.
4. Every iteration step is distinct and does not flow into the next.
5. It takes a lot of time and effort to fix an issue in one unit if it needs to be corrected in all the units.

Incremental Vs Waterfall Model

Incremental software development is better than a waterfall approach for most business, e-commerce, and personal systems. By developing the software incrementally, it is cheaper and easier

to make changes in the software as it is being developed.

Compared to the waterfall model, incremental development has three important benefits:

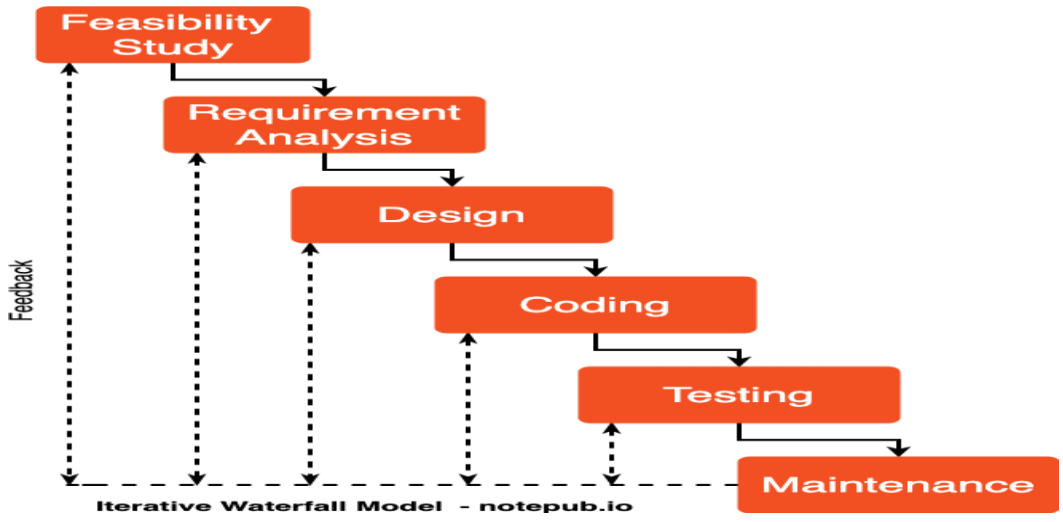
The **cost of accommodating changing** customer requirements is reduced. The amount of analysis and documentation that has to be redone is much less than that's required with waterfall model.
It's easier to get **customer feedback** on the work done during development than when the system is fully developed, tested, and delivered.
More **rapid delivery** of *useful* software is possible even if all the functionality hasn't been included. Customers are able to use and gain value from the software earlier than it's possible with the waterfall model.

Iterative Model:

The Iterative Waterfall Model is a software development approach that combines the sequential steps of the traditional Waterfall Model with the flexibility of iterative design. It allows for improvements and changes to be made at each stage of the development process, instead of waiting until the end of the project. The iterative waterfall model provides feedback paths from every phase to its preceding phases, which is the main difference from the classical waterfall model.

1. When errors are detected at some later phase, these feedback paths allow for correcting errors committed by programmers during some phase.
2. The feedback paths allow the phase to be reworked in which errors are committed and these changes are reflected in the later phases.
3. But, there is no feedback path to the stage – feasibility study, because once a project has been taken, does not give up the project easily.
4. It is good to detect errors in the same phase in which they are committed.
5. It reduces the effort and time required to correct the errors.
6. A real-life example could be building a new website for a small business.

Process of Iterative Waterfall Model



Following are the phases of Iterative Waterfall Model:

1. **Requirements Gathering:** This is the first stage where the business owners and developers meet to discuss the goals and requirements of the website.
2. **Design:** In this stage, the developers create a preliminary design of the website based on the requirements gathered in stage 1.
3. **Implementation:** In this stage, the developers begin to build the website based on the design created in stage 2.
4. **Testing:** Once the website has been built, it is tested to ensure that it meets the requirements and functions properly.
5. **Deployment:** The website is then deployed and made live to the public.
6. **Review and Improvement:** After the website has been live for a while, the business owners and developers review its performance and make any necessary improvements.

This process is repeated until the website meets the needs and goals of the business. Each iteration builds upon the previous one, allowing for continuous improvement and iteration until the final product is complete.

Application of Iterative Waterfall Model:

Below are some application of Iterative Waterfall Model:

1. The essential needs are established, but as time passes, the finer points may become relevant.
2. Programmers have a learning curve to climb when they utilize new technology.
3. The resources needed to complete a large project are constrained, hence on a smaller scale, the automation is more temporary.
4. Very high risk as the project's objective may occasionally alter.

Advantages of Iterative Waterfall Model:

1. Phase Containment of Errors:
2. Collaboration
3. FlexibilityTesting and feedback
4. Scalability
5. MaintenanceEasy to Manage
6. Faster Time to Market
7. Predictable Outcomes
8. Improved Customer Satisfaction
9. Quality Assurance
10. Risk Reduction

Drawbacks of Iterative Waterfall Model:

1. Difficult to incorporate change requests
2. Incremental delivery not supported
3. Overlapping of phases not supported
4. Risk handling not supported
5. Limited customer interactions

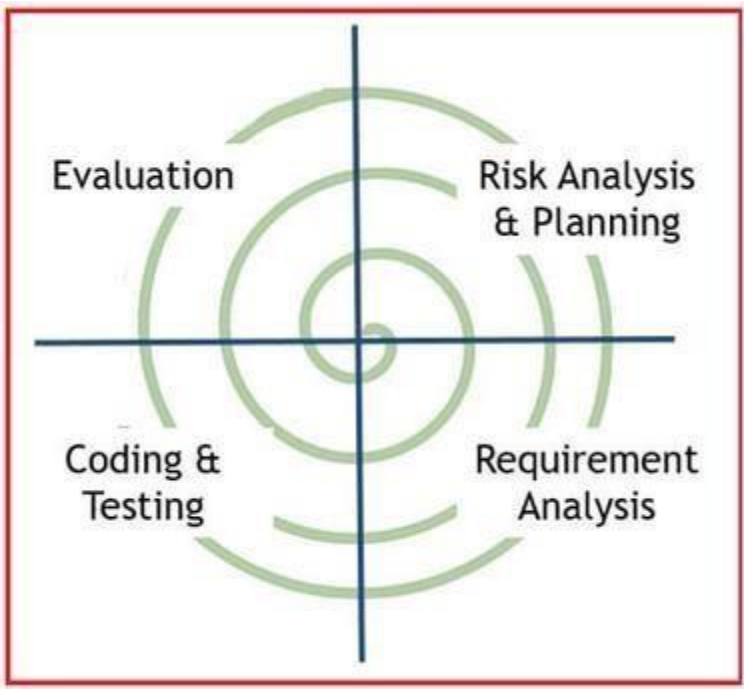
Spiral Model

The Spiral Model is one of the most important Software Development Life Cycle models. The Spiral Model is a combination of the waterfall model and the iterative model. It provides support for **Risk Handling**.

It was designed to include the best features from the waterfall and prototyping models, and introduces a new component; risk-assessment.

What Are the Phases of the Spiral Model?

The Spiral Model is a risk-driven model, meaning that the focus is on managing risk through multiple iterations of the software development process. It consists of the following phases:



1. **Objectives Defined: I**n first phase of the spiral model we clarify what the project aims to achieve, including functional and non-functional requirements.
2. **Risk Analysis:** In the risk analysis phase, the risks associated with the project are identified and evaluated.
3. **Engineering:** In the engineering phase, the software is developed based on the requirements gathered in the previous iteration.
4. **Evaluation:** In the evaluation phase, the software is evaluated to determine if it meets the customer's requirements and if it is of high quality.
5. **Planning:** The next iteration of the spiral begins with a new planning phase, based on the results of the evaluation.

Each phase of the Spiral Model is divided into four quadrants as shown in the above figure. The functions of these four quadrants are discussed below**:**

1. **Objectives determination and identify alternative solutions:** Requirements are gathered from the customers and the objectives are identified, elaborated, and analyzed at the start of every phase. Then alternative solutions possible for the phase are proposed in this quadrant.
2. **Identify and resolve Risks:** During the second quadrant, all the possible solutions are evaluated to select the best possible solution. Then the risks associated with that solution are identified and the risks are resolved using the best possible strategy. At the end of this quadrant, the Prototype is built for the best possible solution.
3. **Develop the next version of the Product:** During the third quadrant, the identified features are developed and verified through testing. At the end of the third quadrant, the next version of the software is available.

4. **Review and plan for the next Phase:** In the fourth quadrant, the Customers evaluate the so-far developed version of the software. In the end, planning for the next phase is started.

Risk Handling in Spiral Model

A risk is any adverse situation that might affect the successful completion of a software project. The most important feature of the spiral model is handling these unknown risks after the project has started. Such risk resolutions are easier done by developing a prototype.

1. The spiral model supports coping with risks by providing the scope to build a prototype at every phase of software development.
2. The **Prototyping Model** also supports risk handling, but the risks must be identified completely before the start of the development work of the project.
3. But in real life, project risk may occur after the development work starts, in that case, we cannot use the Prototyping Model.
4. In each phase of the Spiral Model, the features of the product dated and analyzed, and the risks at that point in time are identified and are resolved through prototyping.
5. Thus, this model is much more flexible compared to other SDLC models.

Example of Spiral Model

Real-Life Example of Spiral Model: Developing an E-Commerce Website

First Spiral – Planning and Requirements: The initial phase involves gathering basic requirements for the e-commerce website, like product listing, shopping cart, and payment options. The team analyzes any risks, such as security or scalability, and creates a small prototype.

Example: The team builds a simple homepage with a basic product catalog to see how users interact with it and identify any design flaws.

Second Spiral – Risk Analysis and Refining the Design: After gathering feedback from the prototype, the next spiral focuses on adding more features and fixing early issues. The team addresses security risks, such as secure payment processing, and tests how well the site handles increasing user traffic.

Example: A basic shopping cart and user registration system are added. The payment system is also tested with dummy transactions to ensure security.

Third Spiral – Detailed Implementation: With more feedback, the team further refines the design, adding advanced features like order tracking, customer reviews, and search functionality. Risks like scalability (handling many users) are re-evaluated, and more testing is conducted.

Example: The website now supports user profiles, product reviews, and real-time inventory updates. The team tests how the system handles large volumes of orders during peak times.

Final Spiral – Full Deployment: The final phase involves full implementation, thorough testing, and launching the e-commerce website to the public. Ongoing risks like system crashes or user feedback are monitored and addressed as needed.

Example: The website goes live with all features, including secure payments, product listings, and order tracking, ready for users to shop online.

Advantages of the Spiral Model

1. Risk Handling
2. Good for large projects
3. Flexibility in Requirements
4. Customer Satisfaction
5. Iterative and Incremental Approach
6. Emphasis on Risk Management
7. Improved Communication
8. Improved Quality

Disadvantages of the Spiral Model

1. Complex
2. Expensive
3. Too much dependability on Risk Analysis
4. Difficulty in time management
5. Complexity
6. Time-Consuming
7. Resource Intensive

Why Spiral Model is called Meta Model?

The Spiral model is called a Meta-Model because it subsumes all the other SDLC models. For example, a single loop spiral actually represents the Iterative Waterfall Model.

The spiral model incorporates the stepwise approach of the Classical Waterfall Model.

The spiral model uses the approach of the Prototyping Model by building a prototype at the start of each phase as a risk-handling technique.

Also, the spiral model can be considered as supporting the Evolutionary model – the iterations along the spiral can be considered as evolutionary levels through which the complete system is

built.

The V – Model :

The V-model is a type of SDLC model where the process executes sequentially in a V-shape. It is also known as the Verification and Validation model. It is based on the association of a testing phase for each corresponding development stage. The development of each step is directly associated with the testing phase. The next phase starts only after completion of the previous phase.

V-Model Design

1. **Requirements Gathering and Analysis:** The first phase of the V-Model is the requirements gathering and analysis phase, where the customer's requirements for the software are gathered and analyzed to determine the scope of the project.

2. **Design:** In the design phase, the software architecture and design are developed, including the high-level design and detailed design.

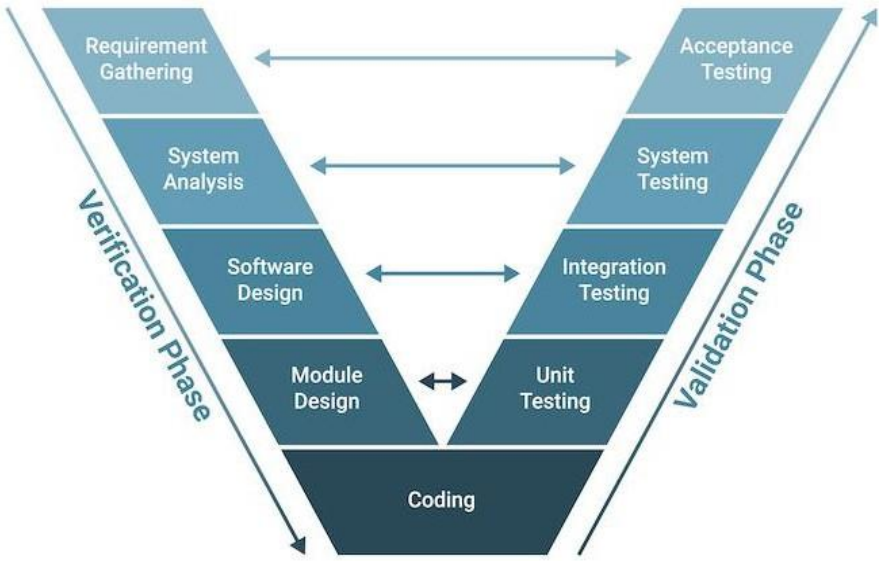
3. **Implementation:** In the implementation phase, the software is built based on the design.

4. **Testing:** In the testing phase, the software is tested to ensure that it meets the customer's requirements and is of high quality.

5. **Deployment:** In the deployment phase, the software is deployed and put into use.

6. **Maintenance:** In the maintenance phase, the software is maintained to ensure that it continues to meet the customer's needs and expectations.

7. **The V-Model is often used in safety:** critical systems, such as aerospace and defence systems, because of its emphasis on thorough testing and its ability to clearly define the steps involved in the software development process.



The following illustration depicts the different phases in a V-Model of the SDLC.
Verification Phases:
It involves a static analysis technique (review) done without executing code. It is the process of evaluation of the product development phase to find whether specified requirements are met.

There are several Verification phases in the V-Model:

Business Requirement Analysis:

This is the first step of the designation of the development cycle where product requirement needs to be cured from the customer's perspective. in these phases include proper communication with the customer to understand the requirements of the customers. these are the very important activities that need to be handled properly, as most of the time customers do not know exactly what they want, and they are not sure about it at that time then we use an **acceptance test design** planning which is done at the time of business requirement it will be used as an **input** for acceptance testing.

System Design:

Design of the system will start when the overall we are clear with the product requirements, and then need to design the system completely. This understanding will be at the beginning of complete under the product development process. these will be beneficial for the future execution of test cases.

Architectural Design:

In this stage, architectural specifications are comprehended and designed. Usually, several technical approaches are put out, and the ultimate choice is made after considering both the technical and financial viability. The system architecture is further divided into modules that each handle a distinct function. Another name for this is High-Level Design (HLD).

At this point, the exchange of data and communication between the internal modules and external systems are well understood and defined. During this phase, integration tests can be created and documented using the information provided.

Module Design:
This phase, known as Low-Level Design (LLD), specifies the comprehensive internal design for every system module. Compatibility between the design and other external systems as well as other modules in the system architecture is crucial. Unit tests are a crucial component of any development process since they assist in identifying and eradicating the majority of mistakes and flaws at an early stage. Based on the internal module designs, these unit tests may now be created.

Coding Phase:
The Coding step involves writing the code for the system modules that were created during the Design phase. The system and architectural requirements are used to determine which programming language is most appropriate.
The coding standards and principles are followed when performing the coding. Before the final build is checked into the repository, the code undergoes many code reviews and is optimized for optimal performance.

Validation Phases**:**
It involves dynamic analysis techniques (functional, and non-functional), and testing done by executing code. Validation is the process of evaluating the software after the completion of the development phase to determine whether the software meets the customer's expectations and requirements.

So, V-Model contains Verification phases on one side of the Validation phases on the other side. The verification and Validation phases are joined by the coding phase in a V-shape. Thus, it is called V-Model.

There are several Validation phases in the V-Model:

Unit Testing:
Unit Test Plans are developed during the module design phase. These Unit Test Plans are executed to eliminate bugs in code or unit level.

Integration testing:
After completion of unit testing Integration testing is performed. In integration testing, the modules are integrated and the system is tested. Integration testing is performed in the Architecture design phase. This test verifies the communication of modules among themselves.

System Testing:
System testing tests the complete application with its functionality, inter-dependency, and communication. It tests the functional and non-functional requirements of the developed application.

User Acceptance Testing (UAT):
UAT is performed in a user environment that resembles the production environment. UAT verifies that the delivered system meets the user's requirement and the system is ready for use in the real world.

Design Phase:

- **Requirement Analysis:** This phase contains detailed communication with the customer to understand their requirements and expectations. This stage is known as Requirement Gathering.

- **System Design:** This phase contains the system design and the complete hardware and communication setup for developing the product.

- **Architectural Design:** System design is broken down further into modules taking up different functionalities. The data transfer and communication between the internal modules and with the outside world (other systems) is clearly understood.

- **Module Design:** In this phase, the system breaks down into small modules. The detailed design of modules is specified, also known as Low-Level Design (LLD).

Testing Phases:

- **Unit Testing:** Unit Test Plans are developed during the module design phase. These Unit Test Plans are executed to eliminate bugs at the code or unit level.

- **Integration testing:** After completion of unit testing Integration testing is performed. In integration testing, the modules are integrated, and the system is tested. Integration testing is performed in the Architecture design phase. This test verifies the communication of modules among themselves.

- **System Testing:** System testing tests the complete application with its functionality, interdependency, and communication. It tests the functional and non-functional requirements of the developed application.

- **User Acceptance Testing (UAT):** UAT is performed in a user environment that resembles the production environment. UAT verifies that the delivered system meets the user's requirement and the system is ready for use in the real world.

Advantages of V-Model

- This is a highly disciplined model and Phases are completed one at a time.
- V-Model is used for small projects where project requirements are clear.
- Simple and easy to understand and use.
- It enables project management to track progress accurately.

Disadvantages of V-Model

- High risk and uncertainty.
- It is not good for complex and object-oriented projects.
- It is not suitable for projects where requirements are not clear and contain a high risk of changing.
- This model does not support iteration of phases.
- It does not easily handle concurrent events.
- Inflexibility: The V-Model is a linear and sequential model, which can make it difficult to adapt to changing requirements or unexpected events.

Specialised Process Models

Specialized process models take on many of the characteristics of one or more of the traditional models. However, these models tend to be applied when a specialized or narrowly defined software engineering approach is chosen.

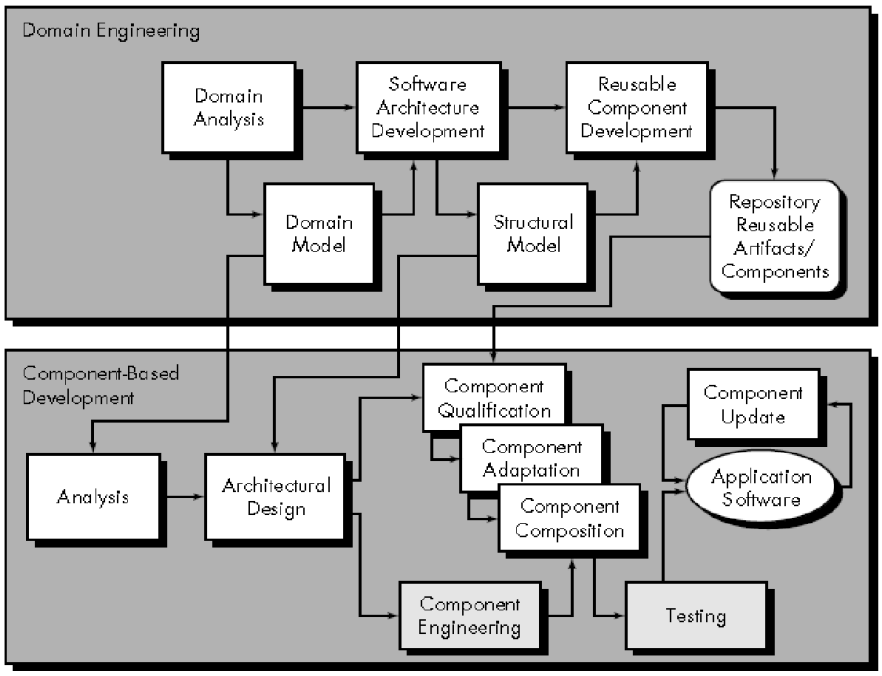
1. Component Based Development
2. Formal Methods Model

1. **Component Based Development** : Commercial off-the-shelf (COTS) software components, developed by vendors who offer them as products, provide targeted functionality with well-defined interfaces that enable the component to be integrated into the software that is to be built. The component-based development model incorporates many of the characteristics of the spiral model. It is evolutionary in nature, demanding an iterative approach to the creation of software. However, the component-based development model constructs applications from prepackaged software component.

Modeling and construction activities begin with the identification of candidate components. These components can be designed as either conventional software modules or object-oriented classes or packages of classes. Regardless of the technology that is used to create the components, the component-based development model incorporates the following steps:

1. Available component-based products are researched and evaluated for the application domain in question.
2. Component integration issues are considered.
3. A software architecture is designed to accommodate the components.
4. Components are integrated into the architecture.
5. Comprehensive testing is conducted to ensure proper functionality

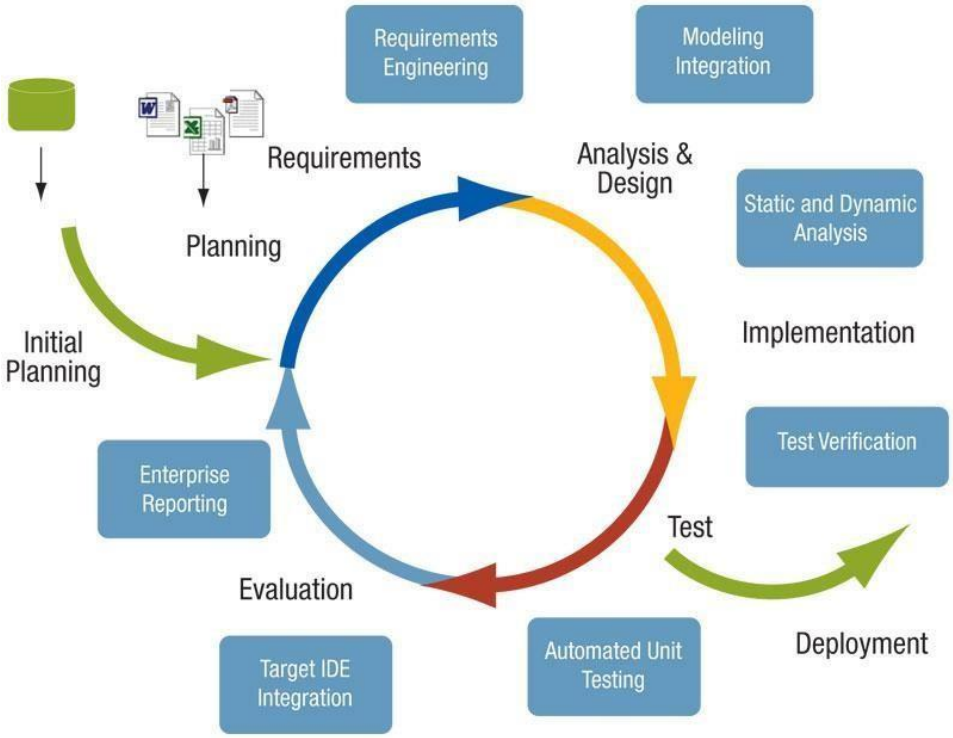
The component-based development model leads to software reuse, and reusability provides software engineers with a number of measurable benefits. software engineering team can achieve a reduction in development cycle time as well as a reduction in project cost if component reuse becomes part of your culture.



2. **Formal Methods Model** : The formal methods model encompasses a set of activities that leads to formal mathematical specification of computer software. Formal methods enable to specify, develop, and verify a computer-based system by applying a rigorous, mathematical notation. A variation on this approach, called cleanroom software engineering is currently applied by some software development organizations. When formal methods are used during development, they provide a mechanism for eliminating many of the problems that are difficult to overcome using other software engineering paradigms.

Ambiguity, incompleteness, and inconsistency can be discovered and corrected more easily, through the application of mathematical analysis. When formal methods are used during design, they serve as a basis for program verification and therefore enable you to discover and correct errors that might otherwise go undetected. The formal methods model offers the promise of defect-free software. There are some of the disadvantages too:

• The development of formal models is currently quite time consuming and expensive.
• Because few software developers have the necessary background to apply formal methods, extensive training is required.
• It is difficult to use the models as a communication mechanism for technically unsophisticated customer.



THE RAD MODEL:

Rapid Application Development (RAD) is an incremental software process model that emphasizes a short development cycle. The RAD model is a "high-speed" adaption of the waterfall model, in which rapid development is achieved by using a component base construction approach.

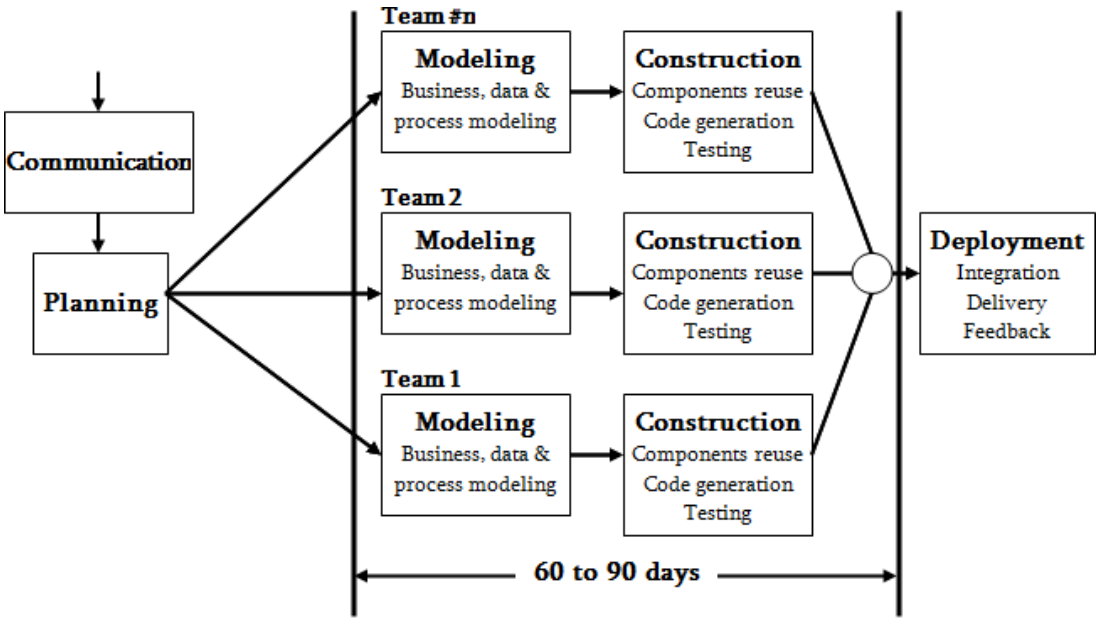


Figure : Flowchart of RAD model

If requirements are well understood and project scope is constrained, the RAD process enables a development team to create a "fully functional system" within a very short time period.

The RAD approach maps into the generic framework activities. Communication works to understand the business problem and the information characteristics that the software must accommodate.

Planning is essential because multiple software teams works in parallel on different system functions.

Modeling encompasses three major phases- business modeling, data modeling and process modeling- and establishes design representation that serve existing software components and the application of automatic code generation.

Deployment establishes a basis for subsequent iterations. The RAD approach has **drawbacks:**

1. For large, but scalable projects, RAD requires sufficient human resources to create the right number of RAD teams.
2. If developers and customers are not committed to the rapid-fire activities necessary to complete the system in a much abbreviated time frame, RAD projects will fail
3. If a system cannot be properly modularized, building the components necessary for RAD will be problematic
4. If high performance is an issue, and performance is to be achieved through tuning the interfaces to system components, the RAD approach may not work; and

RAD may not be appropriate when technical risks are high.

Agile Process View:

Agility is flexibility, it is a state of dynamic, adapted to the specific circumstances. The agile methods refers to a group of software development models based on the incremental and iterative approach, in which the increments are small and typically, new releases of the system are created and made available to customers every few weeks.

Phases in the Agile Model

The agile model is a combination of iterative and incremental process models. The steps involve in agile SDLC models are:

- Requirement gathering
- Design the Requirements
- Construction / Iteration
- Testing / Quality Assurance
- Deployment
- Feedback



Fig. Agile Model

1. **Requirements gathering:**

In this phase, you must define the requirements. You should explain business opportunities and plan the time and effort needed to build the project. Based on this information, you can evaluate technical and economic feasibility**.**

2. **Design the requirements:** When you have identified the project, work with stakeholders to define requirements. You can use the user flow diagram or the high-level UML diagram to show the work of new features and show how it will apply to your existing system.

3. **Construction/ iteration:** When the team defines the requirements, the work begins. Designers and developers start working on their project, which aims to deploy a working product. The product will undergo various stages of improvement, so it includes simple, minimal functionality.
4. **Testing:** In this phase, the Quality Assurance team examines the product's performance and looks for the bug.
5. **Deployment:** In this phase, the team issues a product for the user's work environment.

6. **Feedback:** After releasing the product, the last step is feedback. In this, the team receives feedback about the product and works through the feedback.

Principles of the Agile Model

- To establish close contact with the customer during development and to gain a clear understanding of various requirements, each Agile project usually includes a customer representative on the team. At the end of each iteration stakeholders and the customer representative review, the progress made and re-evaluate the requirements.
- The agile model relies on working software deployment rather than comprehensive documentation.
- Frequent delivery of incremental versions of the software to the customer representative in intervals of a few weeks.
- Requirement change requests from the customer are encouraged and efficiently incorporated.
- It emphasizes having efficient team members and enhancing communications among them is given more importance. It is realized that improved communication among the development team members can be achieved through face-to-face communication rather than through the exchange of formal documents.
- It is recommended that the development team size should be kept small (5 to 9 people) to help the team members meaningfully engage in face-to-face communication and have a collaborative work environment.
- The agile development process usually deploys Pair Programming. In Pair programming, two programmers work together at one workstation. One does coding while the other reviews the code as it is typed in. The two programmers switch their roles every hour or so.

Characteristics of the Agile Process

- Agile processes must be adaptable to technical and environmental changes. That means if any technological changes occur, then the agile process must accommodate them.
- The development of agile processes must be incremental. That means, in each development, the increment should contain some functionality that can be tested and verified by the customer.
- The customer feedback must be used to create the next increment of the process.
- The software increment must be delivered in a short span of time.
- It must be iterative so that each increment can be evaluated regularly.

Advantages of the Agile Model

- Working through Pair programming produces well-written compact programs which have fewer errors as compared to programmers working alone.
- It reduces the total development time of the whole project.
- Agile development emphasizes face-to-face communication among team members, leading to better collaboration and understanding of project goals.
- Customer representatives get the idea of updated software products after each iteration. So, it is easy for him to change any requirement if needed.
- Agile development puts the customer at the centre of the development process, ensuring that the end product meets their needs.

Disadvantages of the Agile Model

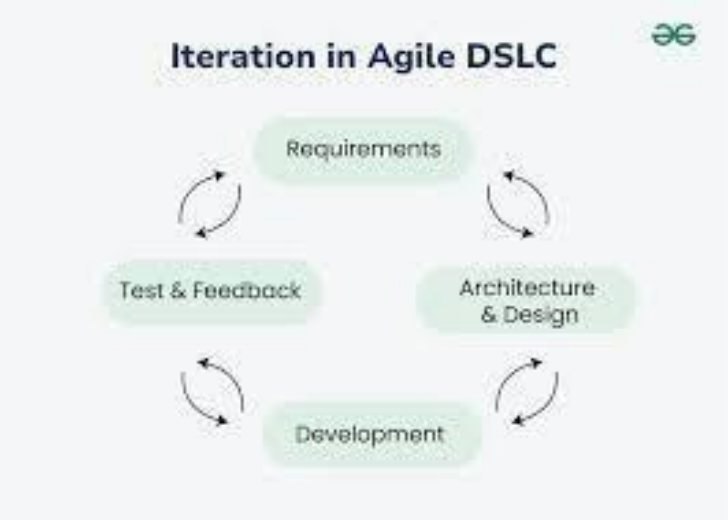
- The lack of formal documents creates confusion and important decisions taken during different phases can be misinterpreted at any time by different team members.
- It is not suitable for handling complex dependencies.
- The agile model depends highly on customer interactions so if the customer is not clear, then the development team can be driven in the wrong direction.

What is the Agile Software Development Life Cycle (Agile SDLC)?

The Agile Software Development Life Cycle (SDLC) is an iterative and incremental software development methodology that prioritizes flexibility, collaboration, and customer feedback. Unlike traditional SDLC models, such as the waterfall model, which completes each step sequentially, the agile SDLC divides the development process into smaller iterations or increments.

The major factors of agile,

- Early customer involvement
- Iterative development
- Self-organizing teams
- Adaptation to change



Agile SDLC Process Flow

1. **Concept:** In concept phase vision of the project, its scope, objectives, and goals are set.
2. **Inception:** This stage is conducted before Construction**.** The major goal of this phase is to complete just enough work to get your team moving in the correct direction.
3. **Iteration/Construction:** The main goal of this phase is to create a consumable solution with enough functionality to satisfy your stakeholders' present needs, or to configure it in the event of a package implementation.
4. **Release:** This stage normally involves final performance testing, security assessments, and documentation updates, as well as user acceptability testing (UAT), which ensures that user demands are satisfied and the product is successful. When the software is complete, it is released into production or made available to users.

   Production: The Production Phase aims to keep systems viable and productive after they have been deployed to the user population

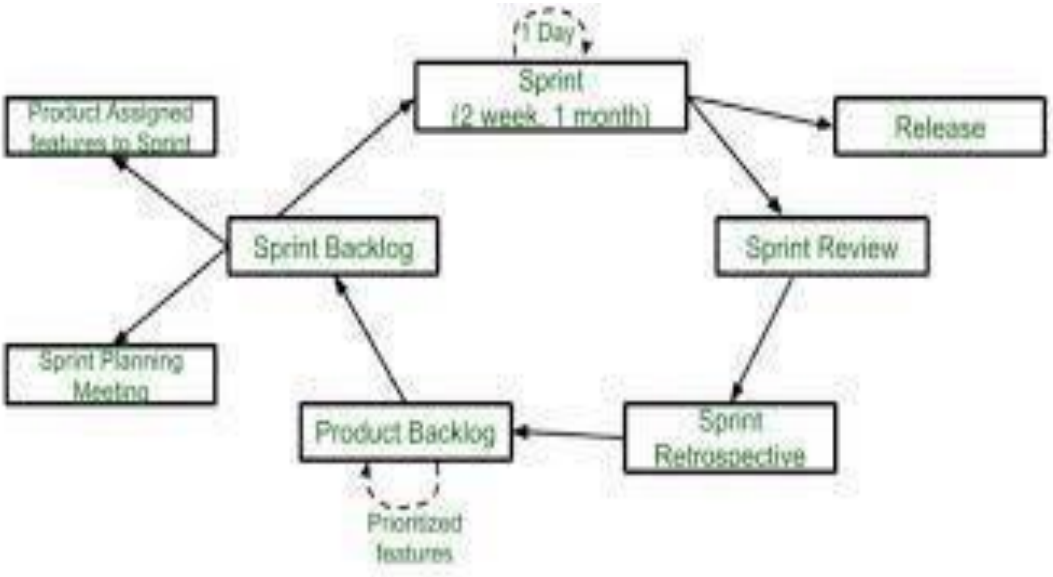
What is a scrum in software development?

Scrum is a management framework that teams use to self-organize tasks and work towards a common goal. It is a framework within which people can address complex adaptive problems while the productivity and creativity of delivering products are at the highest possible value. Scrum is a management framework that teams use to self-organize and work towards a common goal.

- Scrum allows us to develop products of the highest value while making sure that we maintain creativity and productivity.
- The iterative and incremental approach used in scrum allows the teams to adapt to the changing requirements.

Silent features of Scrum

- Scrum is a light-weighted framework
- Scrum emphasizes self-organization
- Scrum is simple to understand
- Scrum framework helps the team to work together
- Lifecycle of Scrum



Sprint: A Sprint is a time box of one month or less. A new Sprint starts immediately after the completion of the previous Sprint. Release: When the product is completed, it goes to the Release stage.

Sprint Review: If the product still has some non-achievable features, it will be checked in this stage and then passed to the Sprint Retrospective stage.

Sprint Retrospective: In this stage quality or status of the product is checked. Product Backlog: According to the prioritize features the product is organized.

Sprint Backlog: Sprint Backlog is divided into two parts Product assigned features to sprint and Sprint planning meeting.

Advantage of Scrum framework

- Scrum framework is fast moving and money efficient.
- Scrum framework works by dividing the large product into small sub-products.
- It's like a divide and conquer strategy
- In Scrum customer satisfaction is very important.
- Scrum is adaptive in nature because it has short sprint.
- As Scrum framework rely on constant feedback therefore the quality of product increases in less amount of time.

Disadvantage of Scrum framework

- Scrum framework does not allow changes into their sprint.
- Scrum framework is not fully described model. If you want to adopt it you need to fill in the framework with your own details like Extreme Programming(XP), Kanban, Dynamic Systems Development Method (DSDM).
- It can be difficult for the Scrum to plan, structure and organize a project that lacks a clear definition.
- The daily Scrum meetings and frequent reviews require substantial resources.

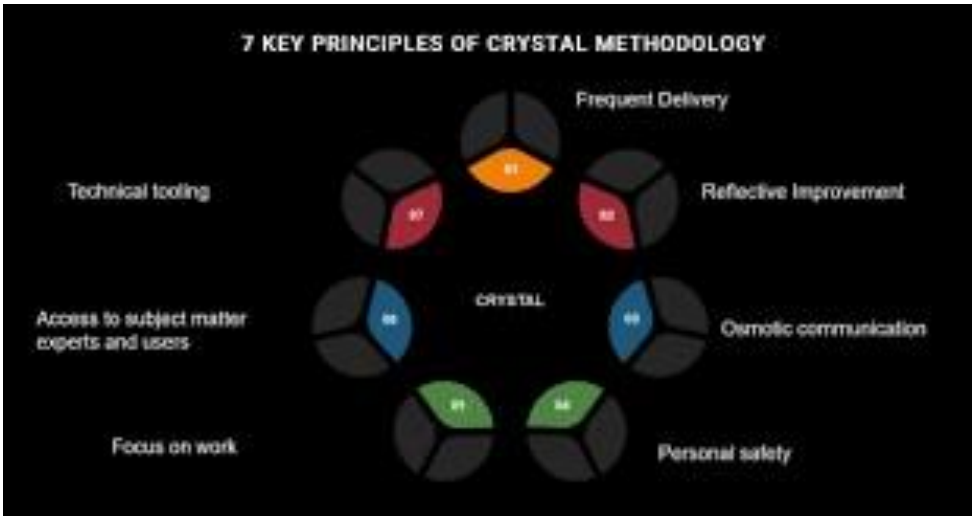
Crystal Method in Agile Development/Framework

Crystal methods are flexible approaches used in Agile software development to manage projects effectively. They adapt to the needs of the team and the project, promoting collaboration, communication, and adaptability for successful outcomes.

The crystal method is an agile framework that is considered a lightweight or agile methodology that focuses on individuals and their interactions. The methods are color-coded to significant risk to human life. It is mainly for short-term projects by a team of developers working out of a single workspace. Among a few Agile Software Development Life Cycle (SDLC) models crystal is considered as one of the Agile SDLC models.

Properties of Crystal Agile Framework

1. **Frequent Delivery:** It allows you regularly deliver the products and test code to real users. Without this, you might build a product that nobody needs.
2. **Reflective Improvement:** No matter how good you have done or how bad you have done. Since there are always areas where the product can be improved, so the teams can implement to improve their future practices.
3. **Osmotic Communication:** Alistair stated that having the teams in the same physical phase is very much important as it allows information to flow in between members of a team as in osmosis.

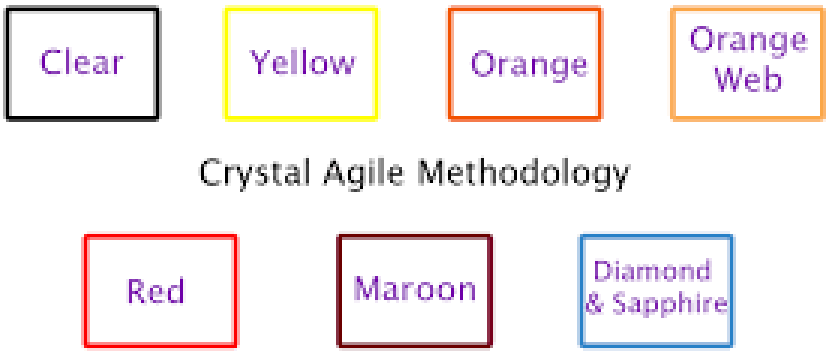


4. **Personal Safety:** There are no bad suggestions in a crystal team, team members should feel safe to discuss ideas openly without any fear.
5. **Focus:** Each member of the team knows exactly what to do, which enables them to focus their attention. This boosts team interaction and works towards the same goal.
6. **Easy access to expert users:** It enhances team communication with users and gets regular feedback from real users.
7. **Technical tooling:** It contains very specific technical tools which to be used by the software development team during testing, management, and configuration. These tools make it enable the team to identify any error within less time.
8. **Continuous learning:** The framework emphasizes on continuous learning, enabling team members to acquire new skills and knowledge, and apply them in their work.

9. **Teamwork:** The framework stresses on the importance of teamwork, promoting collaboration, and mutual support among team members.
10. **Timeboxing:** The framework adopts timeboxing to manage project deadlines, ensuring that the team delivers within set timelines.
11. **Incremental development:** The framework promotes incremental development, enabling the team to deliver working software frequently, and adapt to changes as they arise.
12. **Automated testing:** The framework emphasizes on automated testing, enabling the team to detect and fix bugs early, reducing the cost of fixing errors at later stages.
13. **Customer involvement:** The framework emphasizes on involving customers in the development process, promoting customer satisfaction, and delivering products that meet their needs.
14. **Leadership:** The framework encourages leadership, enabling team members to take ownership of their work and make decisions that contribute to the success of the project.

How does Crystal Methodology Operates?

Till now, we got to know that crystal is a family of various developmental approaches, and it is not a group of prescribed developmental tools and methods. In the beginning, the approach is set by considering the business requirements and the needs of the project. Various methodologies in the Crystal family also known as weights of the Crystal approach are represented by different colors of the spectrum.



Crystal family consists of many variants like Crystal Clear, Crystal Yellow, Crystal Red, Crystal Sapphire, Crystal Red, Crystal Orange Web, and Crystal Diamond.

1. **Crystal Clear-** The team consists of only 1-6 members that is suitable for short-term projects where members work out in a single workspace.
2. **Crystal Yellow-** It has a small team size of 7-20 members, where feedback is taken from Real Users. This variant involves automated testing which resolves bugs faster and redues
3. the use of too much documentation.
4. **Crystal Orange-** It has a team size of 21-40 members, where the team is split according to their functional skills. Here the project generally lasts for 1-2 years and the release is required every 3 to 4 months.
5. **Crystal Orange Web-** It has also a team size of 21-40 members were the projects that have a continually evolving code base that is being used by the public. It is also similar to Crystal Orange but here they do not deal with a single project but a series of initiatives that required programming.
6. **Crystal Red-** The software development is led by 40-80 members where the teams can be formed and divided according to requirements.
7. **Crystal Maroon-** It involves large-sized projects where the team size is 80-200 members and where methods are different and as per the requirement of the software.